

Лекция: «Комбинированный тип данных — запись. Файлы последовательного и прямого доступа. Операции над множествами»

1. Комбинированный тип данных — запись

В Python нет прямого аналога записи (record) из Паскаля, но есть несколько структур данных, которые могут её заменить:

1. Словари (dict) — наиболее гибкий вариант:

```
student = {  
    'name': 'Иванов Иван',  
    'age': 20,  
    'gpa': 4.5,  
    'birth_date': {'day': 15, 'month': 5, 'year': 2003}  
}
```

Доступ к полям

```
print(student['name']) # Иванов Иван
```

```
student['age'] = 21
```

2. Классы — для строгой типизации и методов:

```
class Student:
```

```
    def __init__(self, name, age, gpa, birth_date):
```

```
        self.name = name
```

```
        self.age = age
```

```
        self.gpa = gpa
```

```
        self.birth_date = birth_date
```

Создание объекта

```
student1 = Student('Петров Пётр', 19, 4.2, {'day': 10, 'month': 3, 'year': 2004})
```

```
print(student1.name) # Петров Пётр
```

3. namedtuple — неизменяемая запись:

```
from collections import namedtuple
```

```
Student = namedtuple('Student', ['name', 'age', 'gpa', 'birth_date'])
```

```
student2 = Student('Сидоров Сидор', 20, 3.8, {'day': 25, 'month': 7, 'year': 2002})
```

```
print(student2.name) # Сидоров Сидор
```

4. dataclass (Python 3.7+) — современный подход:

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Student:
```

```
    name: str
```

```
    age: int
```

```
    gpa: float
```

```
    birth_date: dict
```

```
student3 = Student('Кузнецов Алексей', 21, 4.0, {'day': 5, 'month': 12, 'year':  
2001})
```

2. Файлы последовательного доступа

Файлы последовательного доступа в Python обрабатываются построчно или поблочно.

Основные режимы открытия файлов:

- 'r' — чтение;
- 'w' — запись (перезапись);
- 'a' — добавление в конец;
- 'r+' — чтение и запись.

Пример работы с текстовым файлом:

```
# Запись в файл
```

with open('students.txt', 'w', encoding='utf-8') as f:

```
f.write('Иванов Иван, 20, 4.5\n')
```

```
f.write('Петров Пётр, 19, 4.2\n')
```

Последовательное чтение

with open('students.txt', 'r', encoding='utf-8') as f:

```
for line in f: # Чтение построчно
```

```
    name, age, gpa = line.strip().split(' ')
```

```
    print(f'Студент: {name}, возраст: {age}, GPA: {gpa}')
```

Чтение всего файла:

with open('students.txt', 'r') as f:

```
    content = f.read() # Всё содержимое как строка
```

```
    lines = f.readlines() # Список строк
```

Особенности:

- указатель файла автоматически перемещается после каждой операции;
- для больших файлов предпочтительно построчное чтение;
- всегда указывайте кодировку (encoding='utf-8').

3. Файлы прямого доступа

Прямой доступ в Python реализуется через:

1. Бинарные файлы с seek() и tell():

```
import struct
```

Запись данных фиксированной длины

with open('records.dat', 'wb') as f:

```
    record1 = struct.pack('20siif', b'Иванов', 20, 175, 70.5)
```

```
    record2 = struct.pack('20siif', b'Петров', 19, 180, 75.0)
```

```
f.write(record1)
```

```
f.write(record2)
```

Прямой доступ к записи №2 (нумерация с 0)

```
with open('records.dat', 'rb') as f:
```

```
    record_size = 20 + 4 + 4 + 4 # байты: строка + int + int + float
```

```
    f.seek(1 * record_size) # переход ко второй записи
```

```
    data = f.read(record_size)
```

```
    name, age, height, weight = struct.unpack('20siif', data)
```

```
    print(f'{name.decode().strip()}, {age} лет, {height} см, {weight} кг')
```

2. Модуль `mmap` для больших файлов:

```
import mmap
```

```
with open('large_file.dat', 'r+b') as f:
```

```
    mm = mmap.mmap(f.fileno(), 0)
```

```
    # Прямой доступ по смещению
```

```
    mm[100:110] = b'new data'
```

```
    mm.close()
```

Сравнение типов доступа:

Критерий	Последовательный доступ	Прямой доступ
Скорость	Зависит от позиции	Постоянная
Память	Можно обрабатывать большие файлы	Требует загрузки в память или <code>mmap</code>
Удобство	Просто для текста	Сложнее, требует фиксированной структуры
Типичное использование	Логи, конфиги, CSV	Базы данных, кэши

4. Операции над множествами в Python

Создание множеств:

Через set()

```
digits = set([1, 2, 3, 4, 5])
```

Через фигурные скобки

```
letters = {'a', 'b', 'c'}
```

Пустое множество

```
empty_set = set()
```

Операции над множествами:

1. Объединение (| или union()):

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a | b)      # {1, 2, 3, 4, 5}
```

```
print(a.union(b)) # {1, 2, 3, 4, 5}
```

2. Пересечение (& или intersection()):

```
print(a & b)      # {3}
```

```
print(a.intersection(b)) # {3}
```

3. Разность (- или difference()):

```
print(a - b)      # {1, 2}
```

```
print(a.difference(b)) # {1, 2}
```

4. Симметричная разность (^ или symmetric_difference()):

```
print(a ^ b)      # {1, 2, 4, 5}
```

```
print(a.symmetric_difference(b)) # {1, 2, 4, 5}
```

5. Проверка отношений:

```
c = {1, 2}
```

```
d = {1, 2, 3, 4}
```

```
print(c <= d)      # True (подмножество)
print(d >= c)      # True (надмножество)
print(c.issubset(d)) # True
print(d.issuperset(c)) # True
```

6. Проверка принадлежности:

```
print(3 in a)      # True
print(6 not in a)  # True
```

Изменяемые операции (модифицируют исходное множество):

```
a |= b # объединение
a &= b # пересечение
a -= b # разность
a ^= b # симметричная разность
```

Неизменяемые множества (frozenset):

```
frozen = frozenset([1, 2, 3])
# frozen.add(4) # Ошибка!
```

Заключение

В Python:

- **записи** заменяются словарями, классами, namedtuple или dataclass;
- **файлы последовательного доступа** обрабатываются стандартными методами чтения/записи;
- **прямой доступ** реализуется через бинарные файлы с seek() или mmap;
- **множества** предоставляют полный набор математических операций с удобным синтаксисом.

Эти инструменты позволяют эффективно решать задачи хранения и обработки данных любой сложности.